INDIAN INSTITUTE OF TECHNOLOGY PATNA

EC3202: DIGITAL SIGNAL PROCESSING LAB

Experiment No: 01

Topic: Familiarization with Kits and MATLAB

Submitted By:

Name: Abhitesh Shukala
Roll No: 2301EE52
Group No: 01(Wednesday)

<u>Aim 1</u>: To generate basic signals like unit impulse, unit step, unit ramp signal and exponential signals.

Q1. WAP to generate the following signals using MATLAB:

a) Unit step sequence b) Ramp sequence c) Exponential sequence

d) Generate and plot each of the following sequences over the indicated interval.

a. $x(n) = 2\delta(n+2) - \delta(n-4), \quad -5 \le n \le 5.$

b. $x(n) = n[u(n) - u(n-10)] + 10e^{-0.3(n-10)}[u(n-10) - u(n-20)], 0 \le n \le 20.$

c. $x(n) = \cos(0.04\pi n) + 0.2w(n), \ 0 \le n \le 50$, where $w(n)$ is a Gaussian random sequence with zero mean and unit variance.

d. $\tilde{x}(n) = \{..., 5, 4, 3, 2, 1, 5, 4, 3, 2, 1, 5, 4, 3, 2, 1, ...\}; \ -10 \le n \le 9.$

e) Generate the complex-valued signal.

$$x(n) = e^{(-0.1+j0.3)n}, \quad -10 \le n \le 10$$

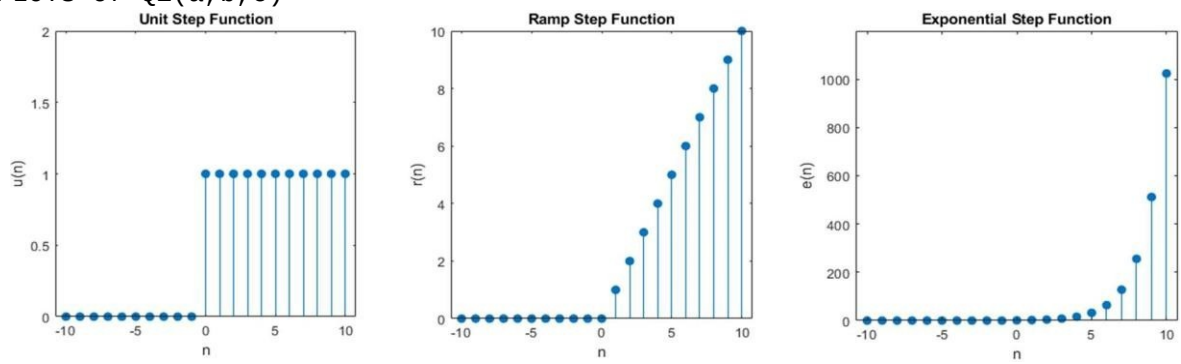
f) Let $x(n) = u(n) - u(n - 10)$. Decompose $x(n)$ into even and odd components.

MATLAB code & plots:

```
%Part a)
Clear;clc;
n = -10:10;
 u = zeros(1,length(n));

 for k = 1:length(n)
     if n(k) >= 0
         u(k) = 1;
     else
         u(k) = 0;
     end
 end
stem(n,u,'filled'); grid on;
title('Unit Step Sequence'); xlabel('n'); ylabel('u(n)');

%Part b)
Clear;Clc;
n = -10:10;
 r = zeros(1,length(n));

 for k = 1:length(n)
     if n(k) >= 0
         r(k) = n(k);
     else
         r(k) = 0;
     end
 end
stem(n,r,'filled'); grid on;
title('Ramp Sequence'); xlabel('n'); ylabel('r(n)');
```

```
%part c)
Clear;clc;
n = -10:10;
 a = 0.8;
 x = zeros(1,length(n));

 for k = 1:length(n)
     if n(k) >= 0
         x(k) = a^n(k);
     else
         x(k) = 0;
     end
 end

 stem(n,x,'filled'); grid on;
title('Exponential Sequence'); xlabel('n'); ylabel('x(n)');
```

PLOTS of Q1(a,b,c)



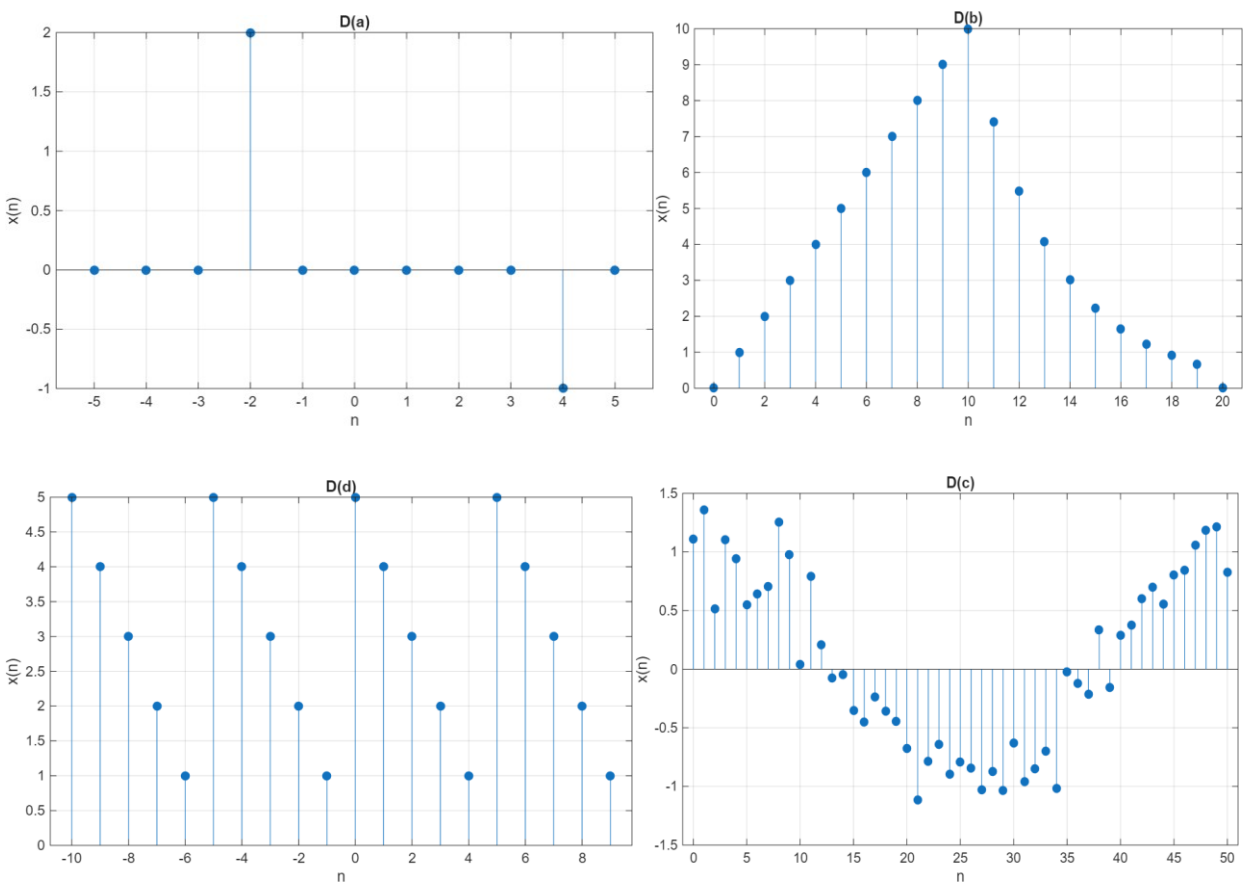
MATLAB code & plots:

```
%Part d(a)
Clear;clc;
n = -5:5;
 x = zeros(1,length(n));

 for k = 1:length(n)
     if n(k) == -2
         x(k) = 2;
     elseif n(k) == 4
         x(k) = -1;
     else
         x(k) = 0;
     end
 end
stem(n,x,'filled');
xlabel('n'); ylabel('x(n)');
title('D(a)');grid on;
```

```
%Part d(b)
Clear;clc;
n = 0:20;
 x = zeros(1,length(n));

 for k = 1:length(n)
     if n(k)>=0 && n(k)<10
         part1 = n(k);
     else
         part1 = 0;
     end
     if n(k)>=10 && n(k)<20
         part2 = 10*exp(-0.3*(n(k)-10));
     else
         part2 = 0;
     end
    x(k) = part1 + part2;
 end

 stem(n,x,'filled');
 xlabel('n'); ylabel('x(n)');
 title('D(b)');
 grid on;

%Part d(c)
Clear;clc;
n = 0:50;
 x = zeros(1,length(n));
 w = zeros(1,length(n));

 for k = 1:length(n)
     w(k) = randn;
     x(k) = cos(0.04*pi*n(k)) + 0.2*w(k);
 end
stem(n,x,'filled');
 xlabel('n'); ylabel('x(n)');
 title('D(c));
 grid on;
```

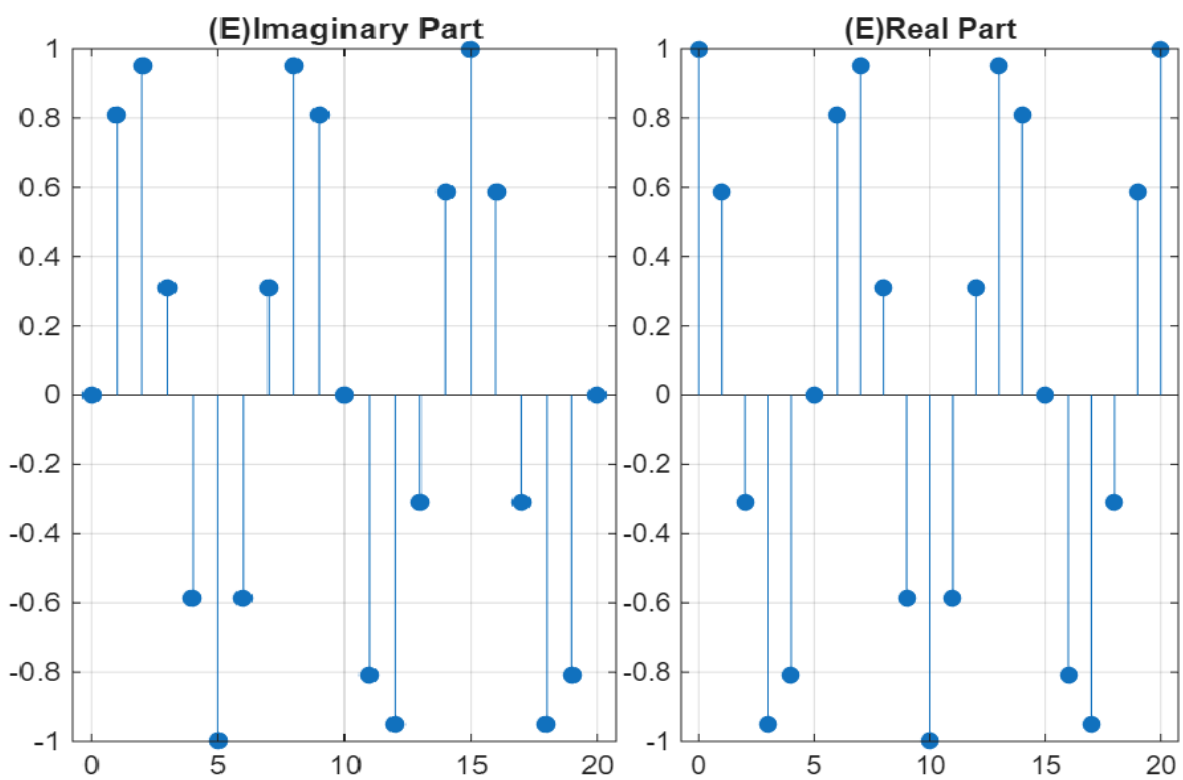
```
%Part  d(d)
Clear;clc;
n = -10:9;
 x = zeros(1,length(n));
 pattern = [5 4 3 2 1];

 for k = 1:length(n)
     index = mod(n(k),5);
     if index < 0
         index = index + 5;
     end
     x(k) = pattern(index + 1);
 end
  stem(n,x,'filled');
 xlabel('n'); ylabel('x(n)');
 title('D(d)');
grid on;
```

PLOTS:-



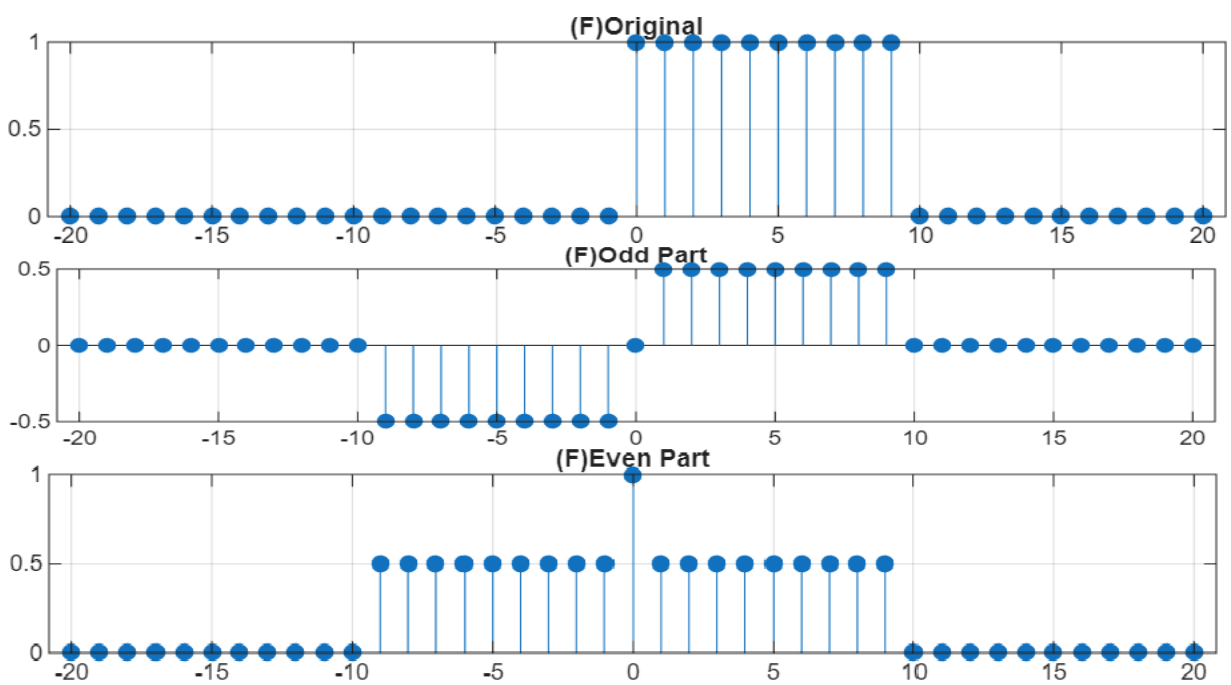
```
%Part e)
Clear;clc;
n = 0:20;
 x = zeros(1,length(n));

for k = 1:length(n)
     x(k) = exp(1j*0.3*pi*n(k));
 end

subplot(1,2,1)
 stem(n,real(x),'filled'); grid on;
 title('(E)Real Part');

subplot(1,2,2)
 stem(n,imag(x),'filled'); grid on;
 title('(E)Imaginary Part');
```

PLOTS:-



```
%Part f)
 n = -20:20;
 x = zeros(1,length(n));
for k = 1:length(n)
     if n(k)>=0 && n(k)<10
         x(k)=1;
     else
         x(k)=0;
     end
 end
x_rev = fliplr(x);
xe = zeros(size(x));
xo = zeros(size(x));
for k = 1:length(x)
     xe(k) = (x(k) + x_rev(k))/2;
     xo(k) = (x(k) - x_rev(k))/2;
 end
subplot(3,1,1)
 stem(n,x,'filled'); title('(F)Original'); grid on;
 subplot(3,1,2)
 stem(n,xe,'filled'); title('(F)Even Part'); grid on;
  subplot(3,1,3)
stem(n,xo,'filled'); title('(F)Odd Part'); grid on;
```

PLOTS:-



Q2. Given the following difference equation

$$y(n) - y(n-1) + 0.9y(n-2) = x(n);\ \forall n$$

a. Calculate and plot the impulse response $h(n)$ at $n = -20,.....,100$.

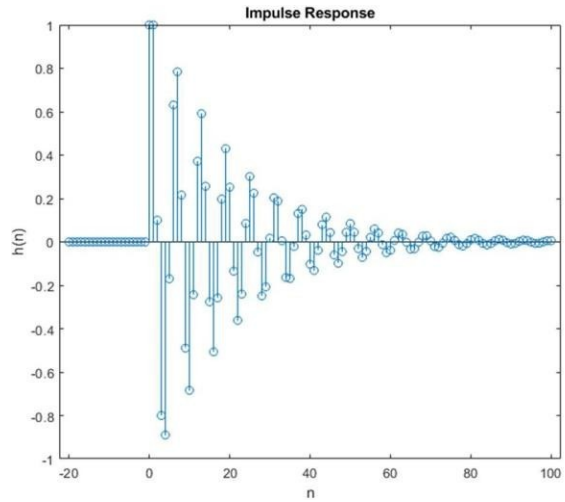
b. Calculate and plot the unit step response $s(n)$ at $n = -20,.....,100$.

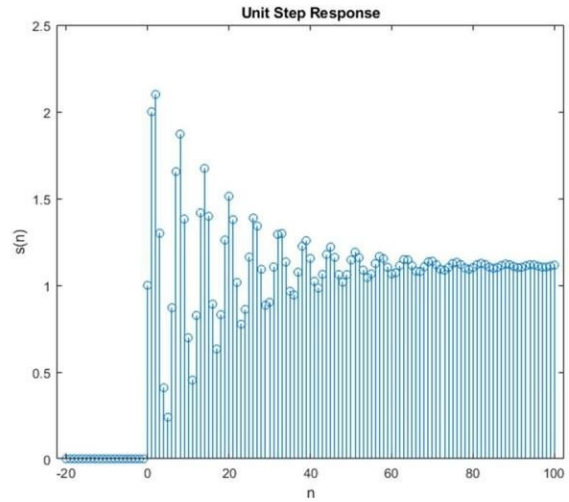
c. Is the system specified by $h(n)$ stable?

MATLAB code & plots:

```
%Part a)
Clear;clc;
n = -20:100;
x = zeros(1,length(n));
y = zeros(1,length(n));

for k = 1:length(n)
    if n(k)==0
        x(k)=1;
    end
end

for k = 3:length(n)
    y(k) = x(k) + y(k-1) - 0.9*y(k-2);
end

stem(n,y,'filled'); grid on;
title('Impulse Response h(n)');


%Part b)
Clear;clc;
x = zeros(1,length(n));
y = zeros(1,length(n));

for k = 1:length(n)
    if n(k)>=0
        x(k)=1;
    end
end

for k = 3:length(n)
    y(k) = x(k) + y(k-1) - 0.9*y(k-2);
end

stem(n,y,'filled'); grid on;
title('Unit Step Response s(n)');
```

Plots:-





c. As the impulse response decays with time and the sum of $h(n)$ is finite. So, the system is stable.

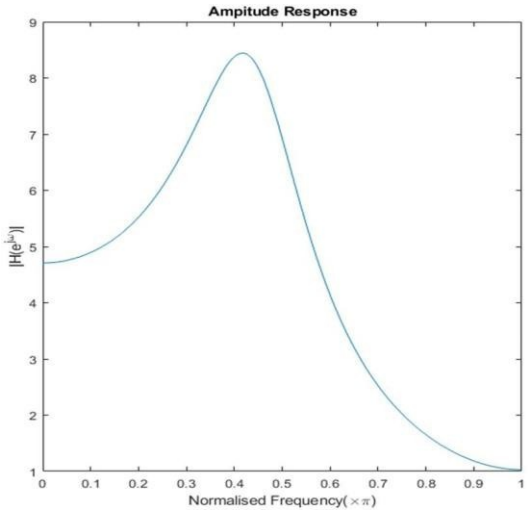
Aim 2: To find the frequency response of a given system given in (Transfer Function/ Differential equation form).

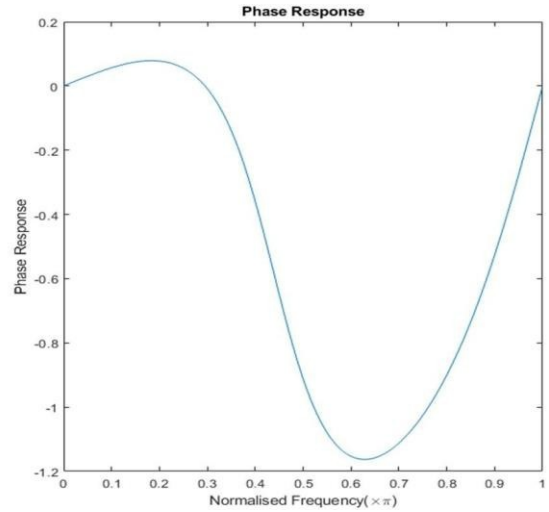
Q3. Find and plot the frequency response (amplitude and phase) of a given system given in (Transfer Function/ Differential equation form).

The difference equation is given as y[n] - 0.25y[n - 1] + 0.45y[n - 2] = 1.55x[n] +1.95x[n - 1] + 2.15x[n]. You can use the inbuilt functions of the MATLAB.

MATLAB code & plots:

```
clear; clc;

n = -50:50;

b = [3.7 1.95] ;
a = [1 -0.25 0.45];
[H, w] = freqz(b, a, 512);

subplot(1, 2, 1);
plot(w/pi, abs(H));
xlabel('Normalised
Frequency(\times\pi)'); ylabel('|
H(e^{j\omega})|'); title('Ampitude
Response');

subplot(1, 2, 2);
plot(w/pi, angle(H));
ylabel('Phase Response');
xlabel('Normalised Frequency(\times\pi)');
title('Phase Response');
```





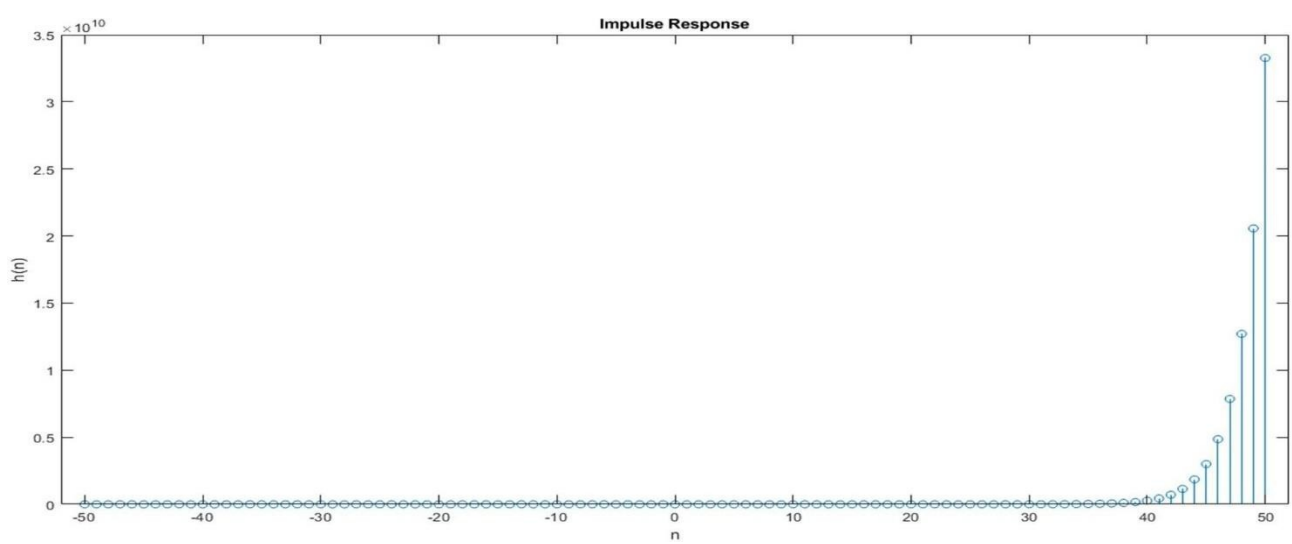
Aim 3: To write a MATLAB program to evaluate the impulse response of the system

Q4. Write a MATLAB program to evaluate the impulse response of the system given as:

$$y(n) = x(n)+0.5x(n - 1) + 0.85x(n - 2) + y(n - 1) + y(n - 2).$$

MATLAB code & plot:

```
clc; clear;

n = -50:50;
x = double(n == 0);
b = [1 0.5 0.85] ;
a = [1 -1 -1];
h = filter(b, a, x);

stem(n, h);
title('Impulse Response');
xlabel('n');
ylabel('h(n)');
```



Aim 4: To write a "C" program to generate various wave forms (Sine wave, Square wave) graphically with using Code Composer Studio (CCS-7.4) IDE and the DSP TMS 320C6713 DSK Kit.

REQUIREMENT: 1. Code Composer Studio (Software installed in Desktop PC)

2. DSP TMS 320C6748 DSP Kit

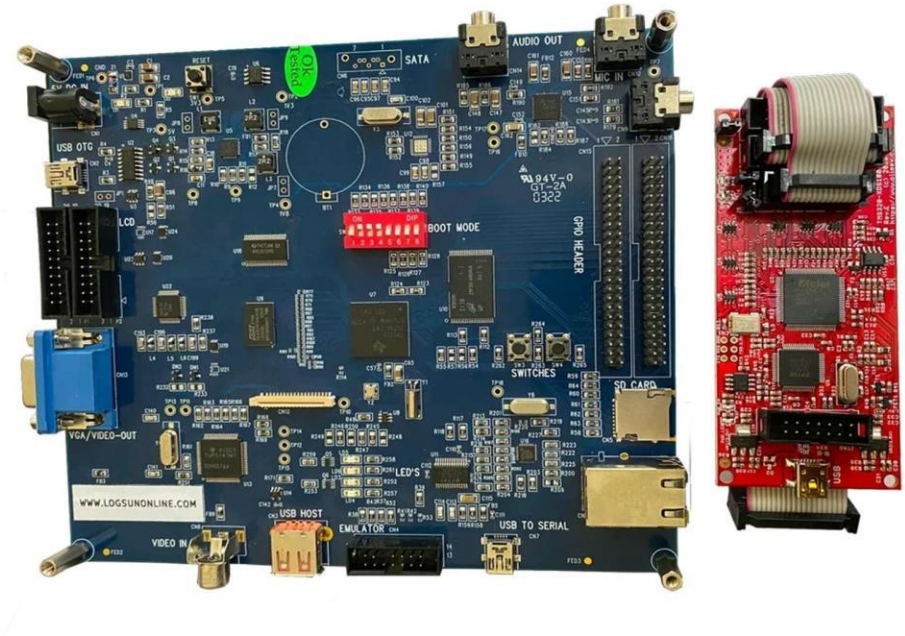
Introduction to the Code Composer Studio

CCS is the DSP industry's first fully integrated development environment (IDE) with DSP specific functionality. With a familiar environment like MS-based C++ TM, Code Composer lets you edit, build, debug, profile & manage projects from a single unified environment. Other unique features include graphical signal analysis, injection/extraction of data signals via I/O, multi-processor debugging, automated testing and customization via C-interpretive scripting language & much more.

Some of the features are:

1. IDE
2. Debug IDE
3. Advanced watch windows
4. Integrated editor
5. File I/O, Probe Points and graphical algorithm scope probes
6. Advanced graphical analysis
7. Interactive profiling
8. Automated testing & customisation via scripting

9. Visual project management system
10. Compile in background while editing & debugging
11. Multi-processor debugging
12. Help on target DSP



DSP (TMS320C6748) Development board with XDS100V3 USB Emulator

PROCEDURE:

1. Once the PC is booted, turn ON the power of DSP kit. Now connect the USB of DSP starter kit to the PC.
2. Start CCS
3. To create project: **File/New/CCS Project**
   - i) Select C674x Floating-point DSP
   - ii) Target - TMS320C6748
   - iii) Connection – Texas Instruments XDS100v3 USB Emulator
   - iv) Project creation name
   - v) Complier version – TIv7.4.8
   - vi) Finish
   - vii) On the top left **name [Active Debug]** will appear and **main.c** window will open.

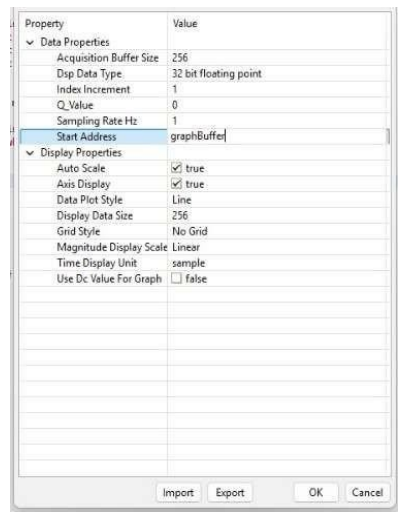
4. Now we are supposed to write a program based on ‘C’.

Code:
```
#include <math.h>

#define BUFFER_SIZE 256
float graphBuffer[BUFFER_SIZE];
float squareBuffer[BUFFER_SIZE];
float theta = 0;

void main(void) {
    int i;
    while(1) {
        for(i = 0; i < BUFFER_SIZE; i++) {
            graphBuffer[i] = sin(theta);
            if(graphBuffer[i] > 0)
               squareBuffer[i] = 1.0;
            else
               squareBuffer[i] = -1.0;
            theta += 0.1;
        }
        theta = 0;
    }
}
```

5. To build: **Project/Build Project**

6. To debug: **Run/Debug**

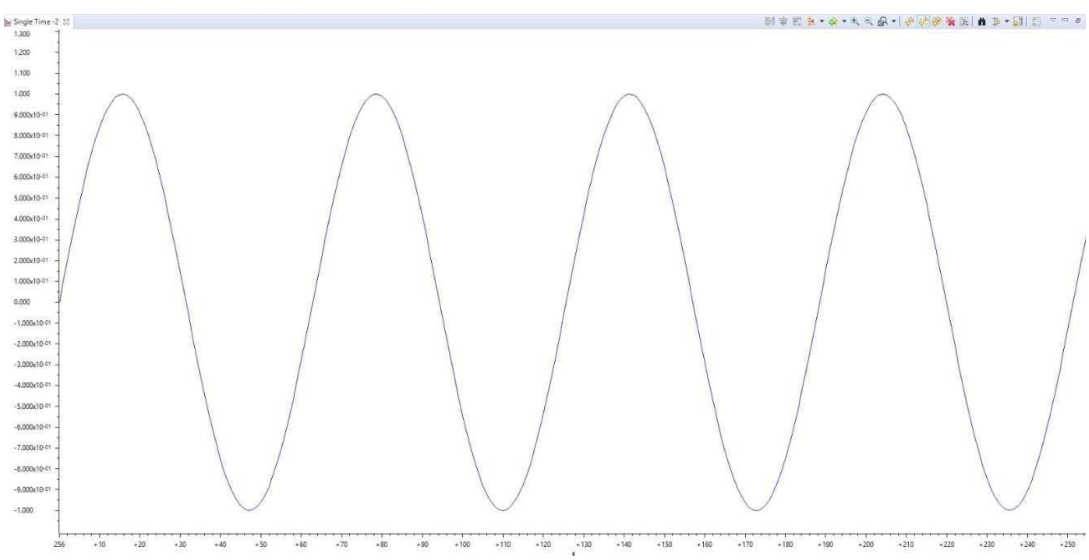
7. Tools property:



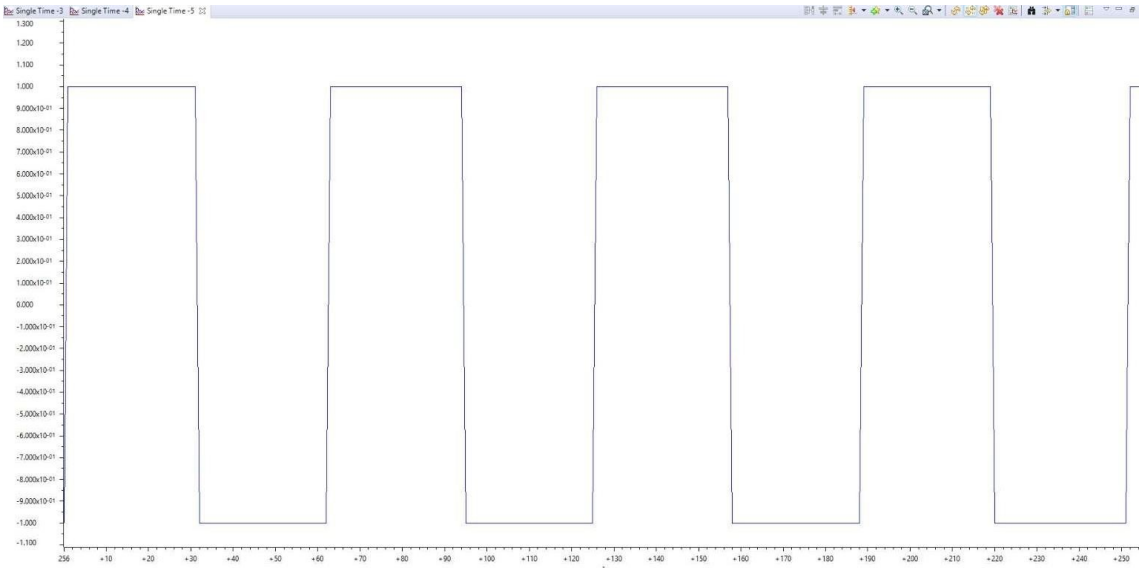
To run a project:

Plots:

Sine wave:



Square wave:



<u>Result:</u> We have successfully done each part of the experiment and matched the data with calculated values and understood the operation.